

Assignment 2: IoT/Cloud Computing Design Project

Objective

Design, implement, and evaluate an Internet of Things (IoT) solution that integrates with cloud services to address a real-world problem, with a strong focus on leveraging cloud-based services for scalability, reliability, and data processing. The project must include a report and a video presentation not exceeding 15 minutes.

Tasks

1. Problem Identification and Research:

- Select a real-world problem that can be addressed using IoT and cloud computing.
- Describe the problem in detail and outline the key objectives of your solution.
- Research at least 2–3 existing IoT architectures and cloud computing methodologies relevant to your project.
- Explain why an IoT-cloud solution is appropriate, emphasizing cloud-specific benefits such as scalability, serverless computing, and cost-efficiency.

2. System Design:

- Define the system architecture, specifying hardware components (e.g., sensors, microcontrollers, actuators).
- Create a detailed block diagram showing interactions between IoT devices, network components, and cloud services, clearly illustrating cloud service data flows (e.g., AWS IoT Core to Lambda).
- Choose appropriate cloud services such as AWS IoT Core, Lambda, DynamoDB, or S3.
- Document communication protocols (e.g., MQTT, HTTP) and data flow, including a rationale for protocol selection.

3. Prototype Implementation:

- Assemble the IoT device using platforms like Raspberry Pi or Arduino.
- Develop firmware/software to collect sensor data and transmit it to the cloud.
- Local code on IoT devices is permitted for data collection and transmission; however, **all backend services**, including data ingestion,

storage, processing, analytics, and user authentication, **must be implemented using cloud-based services.**

- Set up a cloud backend using exclusively cloud-based services to ensure scalability, reliability, and modern IoT solution design. The backend should include:
 - **Data Ingestion:** Configure a scalable pipeline to receive data from IoT devices using technologies like AWS IoT Core, MQTT, or HTTP endpoints, or equivalent alternatives.
 - **Data Storage:** Select an appropriate cloud storage solution (e.g., AWS S3, DynamoDB, RDS) based on data type and access frequency, justifying your choice.
 - **Data Processing:** Implement functions or services (e.g., AWS Lambda, EC2) to process, transform, and analyze incoming data, with robust error handling and reliability testing.
 - **Security & Access Control:** Apply security measures such as IAM roles, security groups, and encryption mechanisms.
 - **Data Analytics (Optional Bonus):** Use cloud-based analytics tools such as AWS CloudWatch, QuickSight, or optional AI/ML-based insights to extract insights from collected data.
- Implement a user interface (web or mobile) that includes:
 - Real-time data visualization through charts, graphs, or tables.
 - Interactive controls to adjust system settings or trigger actions remotely.
 - Secure login and authentication features for user access management.
 - Notifications or alerts based on predefined conditions (e.g., email, SMS, push notifications) (Optional Bonus Task).

4. Video Demonstration (must not exceed 15 minutes):

- A walkthrough of the system's functionality, including:
 - Data ingestion, storage, processing, and analytics, with clear demonstration of cloud service interactions (e.g., data flow through AWS IoT Core to DynamoDB).
 - User interface with real-time visualization, controls, authentication, and notifications.

- Explanation of how the system meets project objectives and its real-world applicability.
- Structure the presentation to cover: 2–3 minutes for problem identification, 5–7 minutes for system demonstration, and 3–5 minutes for design explanation.
- Ensure a professional and clear presentation.

5. **Project Report (recommended 10–20 pages):**

- Submit a detailed report covering:
 - **Problem Definition:** Clear problem statement and objectives.
 - **Research Findings:** Analysis of existing solutions and methodologies.
 - **System Architecture and Design:** Detailed architecture and component selection.
 - **Implementation Steps:** Cloud backend and UI implementation details.
 - **Source Code Explanations:** Relevant code snippets with commentary.
 - **Benefits of Cloud Integration:** How cloud services enhance scalability, reliability, and cost-efficiency.
 - Include diagrams, screenshots, and justifications under relevant subheadings.
 - Provide setup instructions, a troubleshooting guide addressing cloud-specific issues (e.g., AWS credential errors, API Gateway failures), and a user manual for easy deployment and testing.

Technology and Tools

Students may choose suitable IoT hardware, cloud platforms, and programming languages. Students are **not required to use AWS**. Equivalent services on **Azure or Google Cloud** are acceptable, provided choices are clearly justified. Students choosing Arduino are expected to independently learn required device-side programming; assessment focuses on cloud integration rather than hardware-specific implementation details. Suggested options include:

- **Hardware:** Raspberry Pi, Arduino.

- **Cloud Platforms:** AWS IoT Core, Lambda, DynamoDB, S3, QuickSight; Google Cloud IoT, Azure IoT Hub.
- **Protocols:** MQTT (e.g., AWS IoT Core, Mosquitto), HTTP.
- **Tools:** Python, Node.js, Chart.js for visualization.

Deliverables

- **Source Code (ZIP file):**
 - All code, including:
 - IoT device firmware/software for cloud communication.
 - Cloud backend code for data ingestion, storage, processing, analytics, and security.
 - Web/mobile UI code with visualization, controls, authentication, notifications, and data export.
 - Include a README with setup instructions and dependencies.
- **Comprehensive Report (PDF or Word):**
 - Structured document covering all marking criteria sections, with diagrams, screenshots, and code snippets.
 - Include a “Benefits of Cloud Integration” section.
 - Setup instructions, troubleshooting guide, and user manual.
- **Video Demonstration (MP4, 1080p, not exceeding 15 minutes):**
 - Showcase system functionality, cloud interactions, and real-world applicability.
 - Upload to a streaming service (e.g., YouTube, Stream) with an accessible link.
- **Generative AI Use Statement:**
 - Declare any AI tools used, specifying their contributions (e.g., “Used ChatGPT for debugging MQTT code”).

Marking Criteria (Total 100 Marks)

1. **Problem Identification and Research (10 Marks):**
 - Clear problem definition and objectives (5 Marks)
 - Research existing solutions and methodologies (3 Marks)

- Explanation of why an IoT-cloud approach is suitable, emphasizing cloud benefits (2 Marks)

2. System Design (10 Marks):

- Define and justify hardware and cloud service selection (3 Marks)
- System block diagram illustrating all components and cloud interactions (5 Marks)
- Data flow and communication protocols with rationale (2 Marks)

3. Prototype Implementation (40 Marks):

- **Cloud Backend (25 Marks):** Must exclusively utilize cloud-based services:
 - Data Ingestion (6 Marks): Scalable pipeline (e.g., AWS IoT Core, MQTT, HTTP).
 - Data Storage (6 Marks): Cloud storage with justified schema design.
 - Data Processing (8 Marks): Cloud-based processing (e.g., Lambda) with error handling.
 - Security & Access Control (5 Marks): IAM roles, encryption, authentication.
 - Data Analytics (Bonus – up to 5 Marks, non-additive): Cloud-based analytics (e.g., CloudWatch, QuickSight).
- **User Interface (10 Marks):**
 - Real-time Data Visualization (3 Marks)
 - Interactive Controls (3 Marks)
 - Secure Login & Authentication (4 Marks)
 - Notifications & Alerts (Bonus – up to 5 Marks, non-additive)
- **Code Quality (5 Marks):** Modularity, documentation, and adherence to best practices.

4. Video Demonstration (20 Marks):

- Effective demonstration of system functionality, including cloud backend and UI (10 Marks)

- Clear and professional presentation covering problem, design, and implementation (10 Marks)

5. **Comprehensive Report (20 Marks):**

- Clarity, depth, and professionalism in documenting problem, design, and implementation, with a “Benefits of Cloud Integration” section (15 Marks)
- Well-structured setup instructions, troubleshooting guide, and user manual (5 Marks)

Note: Note: Bonus marks are **non-additive** and may be used to offset marks lost in other sections. The final mark is capped at **100**. Failure to submit the Generative AI Use Statement results in a 50-mark deduction.

Submission Format

- **Report:** PDF or Word.
- **Video:** MP4, 1080p, uploaded to YouTube/Stream with an accessible link.
- **Source Code:** ZIP with clear directory structure and README.
- **AI Use Statement:** Detailed declaration of AI tool usage.

Checklist Before Submission

- All files are labeled with your name or student ID.
- Video link is accessible without password protection.
- Code runs error-free on a clean setup.
- Cloud services (e.g., AWS IAM roles, DynamoDB tables) are tested and functional.
- Video tested on a different device to confirm accessibility.

Warning: Potential Costs

Some AWS services (e.g., QuickSight, DynamoDB, S3, EC2) may incur charges after free-tier limits (e.g., DynamoDB: 25 GB free storage monthly). QuickSight starts to charge post 30-day free trial. To avoid unnecessary charges:

- Monitor the AWS Billing Dashboard regularly.
- Delete resources after submission:
 - DynamoDB: Delete tables (e.g., via AWS Console: DynamoDB > Tables > Delete).
 - Lambda: Delete functions (Lambda > Functions > Delete).

- QuickSight: Delete datasets and analyses (QuickSight > Manage > Delete).
- S3: Empty and delete buckets (S3 > Buckets > Empty > Delete).
- IoT Core: Delete rules and things (IoT Core > Manage > Delete).